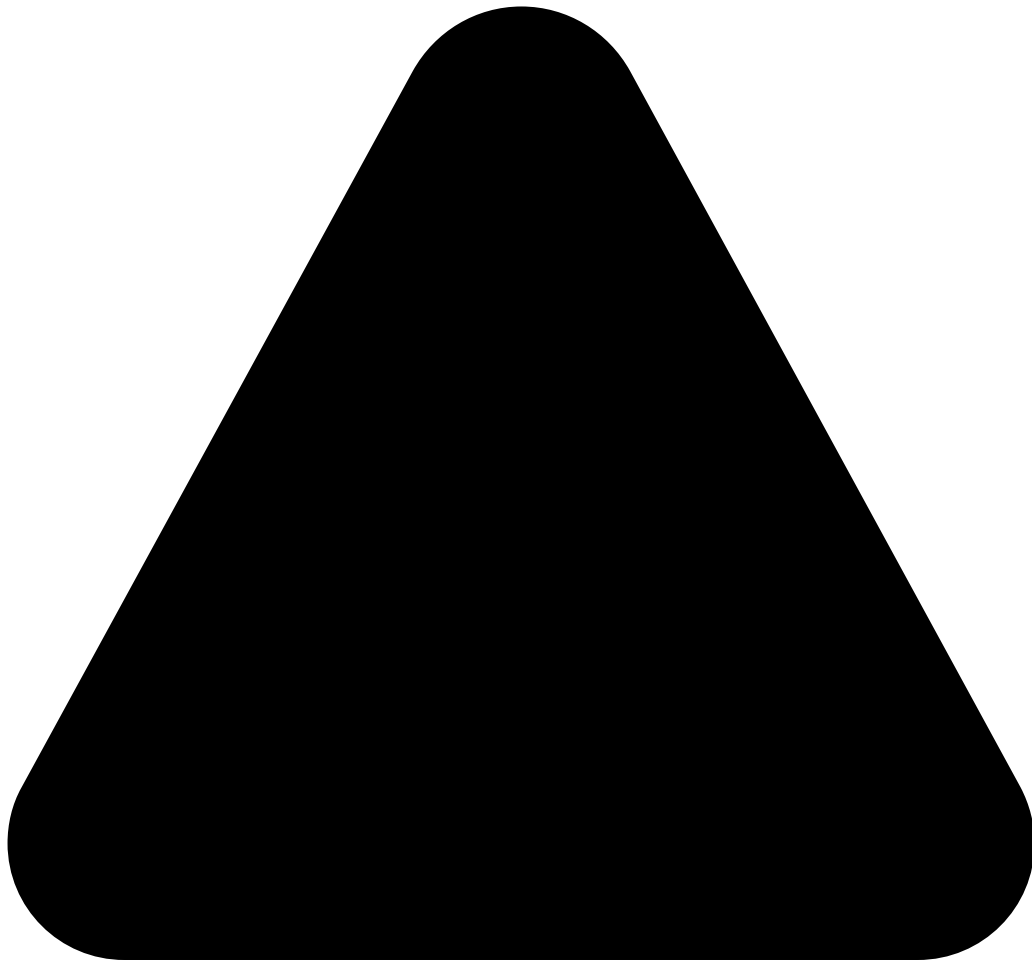


A light gray rectangular box with a thin black border, containing the text "What's New".

What's New

Lab Content

- [Lab overview](#)
- [Start lab](#)
- [Task 1: Create an anomaly detection alarm](#)
- [Task 2: Retrieve metrics and alarm details with the AWS CLI](#)
- [Task 3: Post a custom metric to CloudWatch metrics](#)
- [Task 4: Observe anomaly detection alarm state](#)
- [Knowledge Check](#)
- [Conclusion](#)
- [End lab](#)
- [Additional resources](#)



Important Information

You are using a training account with limited access to Amazon Bedrock. Due to the service quotas set for Requests per Minute (RPM), Tokens per Minute (TPM) and Tokens per Day (TPD) , you may encounter throttling issues. To mitigate these issues:

1. Execute code blocks one at a time.
2. If you receive an error, wait a few seconds before retrying.
3. Avoid running multiple operations concurrently.
4. If throttling persists, wait 2 minutes before running any code that interacts with Bedrock.
5. If the problem continues after waiting, you may need to end the lab and start again.

We understand this is not ideal and are working hard to implement a permanent solution. Following these steps will help ensure a smoother experience as you explore Amazon Bedrock's capabilities. In the interim, please provide the details of any problems you encounter using the [AWS Training and Certification Contact Form](#).

DevOps and AI on AWS: CloudWatch Anomaly Detection

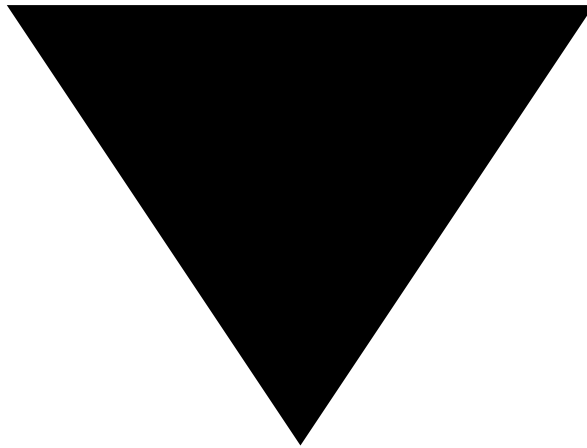
SPL-TF-200-DOCWAD-1 - Version 1.0.2

© 2026 Amazon Web Services, Inc. or its affiliates. All rights reserved. This work may not be reproduced or redistributed, in whole or in part, without prior written permission from Amazon Web Services, Inc. Commercial copying, lending, or selling is prohibited. All trademarks are the property of their owners.

Note: Do not include any personal, identifying, or confidential information into the lab environment. Information entered may be visible to others.

Corrections, feedback, or other questions? Contact us at [AWS Training and Certification](#).

Lab overview



Show details

In this lab, you work with Amazon CloudWatch anomaly detection to monitor and detect unusual patterns in application metrics. You enable anomaly detection for a metric, configure alarms based on expected values, and use both the AWS Command Line Interface (AWS CLI) and a Python script to retrieve and post custom metrics. Through these activities, you learn how CloudWatch can automatically identify and alert on metric anomalies in your applications.

Objectives

By the end of this lab, you should be able to do the following:

- Enable CloudWatch anomaly detection for a metric.
- Post and retrieve CloudWatch metrics.
- Configure an Amazon CloudWatch anomaly detection alarm.

Technical knowledge prerequisites

This lab requires the use of the Chrome internet browser. Using a different browser may cause issues in the developer environment.

Icon key

Various icons are used throughout this lab to call attention to different types of instructions and notes. The following list explains the purpose for each icon:

- **Caution:** Information of special interest or importance (not so important to cause problems with the equipment or data if you miss it, but it could result in the need to repeat certain steps).
- **Command:** A command that you must run.
- **Expected output:** A sample output that you can use to verify the output of a command or edited file.
- **Note:** A hint, tip, or important guidance.
- **Task complete:** A conclusion or summary point in the lab.
- **Warning:** An action that is irreversible and could potentially impact the failure of a command or process (including warnings about configurations that cannot be changed after they are made).

Lab environment

In this lab, you use the AWS Management Console and AWS CLI to work with Amazon CloudWatch features. You also use a Python script to post custom metrics to Amazon CloudWatch.

Services used in this lab

Amazon CloudWatch metrics

Amazon CloudWatch provides real-time monitoring of your Amazon Web Services (AWS) resources and applications. Through metrics collection and tracking, you can measure, analyze, and create alarms for various performance indicators. In this lab, you'll work specifically with custom metrics and anomaly detection, allowing you to automatically identify unusual patterns in your application's behavior.

You can publish your own metrics to CloudWatch using the AWS CLI or an API.

Amazon CloudWatch anomaly detection

When you enable anomaly detection for a metric, CloudWatch applies statistical and machine learning algorithms. These algorithms continuously analyze metrics of systems and applications, determine normal baselines, and surface anomalies with minimal user intervention.

AWS services not used in this lab

AWS service capabilities used in this lab are limited to what the lab requires. Expect errors when accessing other services or performing actions beyond those provided in this lab guide.

Start lab

1. To launch the lab, at the top of the page, choose **Start Lab**.

Caution: You must wait for the provisioned AWS services to be ready before you can continue.

2. To open the lab, choose



Open Console

You are automatically signed in to the AWS Management Console in a new web browser tab.

Warning: Do not change the **Region** unless instructed.

Common sign-in errors

Error: Choosing Start Lab has no effect

In some cases, certain pop-up or script blocker web browser extensions might prevent the **Start Lab** button from working as intended. If you experience an issue starting the lab:

- Add the lab domain name to your pop-up or script blocker's allow list or turn it off.
 - Refresh the page and try again.
-

Task 1: Create an anomaly detection alarm

An application is posting a custom metric to CloudWatch metrics with a count of the currently logged in users.

In this task, you enable anomaly detection and create an alarm for the currently logged in users metric. The metric follows a predictable pattern of values over a 24 hours period. CloudWatch anomaly detection applies statistical and machine learning algorithms to determine normal baselines, and surface anomalies.

A CloudWatch anomaly detection alarm is created based on a metric's expected value. Instead of configuring a static threshold for the alarm, the alarm compares the metric's value to the expected value based on the anomaly detection model.

3. At the top of the AWS Management Console, in the search bar, search for and choose

☐ CloudWatch

4. In the left navigation pane, expand the **Metrics** section, choose **All metrics**.

5. Under **Custom namespaces**, choose **TravelApplication**.

Note: CloudWatch uses namespaces as a container for related metrics. AWS services and customer application use namespaces to organize related metrics.

6. Choose **Metrics with no dimensions**.

Note: CloudWatch metric dimensions are optional key-value pairs used to filter and categorize metrics.

7. Select the checkbox for the **UserLogins** metric.

8. Choose the **Graphed metrics** tab.

9. In the **Period** column, select **1 minute**.

Note: The metric graph now shows more detail as the metric is being posted every 60 seconds.

10. To enable anomaly detection, in the **Actions** column, choose the anomaly icon.

 Anomaly icon

Image description: The diagram depicts the icon options in the metrics actions column.

11. To create an alarm, in the **Actions** column, choose the create alarm bell icon.

 Create alarm icon

Image description: The diagram depicts the icon options in the metrics actions column.

12. On the **Specify metric and conditions** page, leave the default options, choose **Next**.

13. On the **Configure actions** page, in the **Notification** section, choose **Remove**.

Note: A CloudWatch alarm can trigger an action, like sending a notification, when the alarm state changes. For this lab the alarm state is only observed in the console.

14. Choose **Next**.

15. On the **Add name and description** page, for **Alarm name** enter

logins-alarm

16. Choose **Next**.

17. On the **Preview and create** page, choose **Create alarm**.

Note: The alarm will initially appear in a *Insufficient data* state. The alarm will take from a few minutes to gather enough data. Continue on with the next tasks. The final task returns to this alarm.

Task complete: You successfully created an anomaly detection alarm.

Task 2: Retrieve metrics and alarm details with the AWS CLI

In this task, you use the AWS CLI to retrieve details about the anomaly alarm and related metrics.

18. From the panel to the left of these lab instructions, copy the **LabWorkspaceURL** URL value and paste it into a new browser tab.

A page displays that prompts you to enter a password.

19. Copy the **LabWorkspacePassword** value from the panel to the left of these instructions, paste it in the **Password** field and login.

A **Visual Studio Code** (VS Code) IDE displays.

Note: Prior experience with VS Code IDE is not required.

20. Select the **Notifications** icon in the very bottom right corner of the VS Code IDE (the bell icon) to clear any notifications that appeared when you opened the VS Code IDE.

The lower pane includes the following five tabs:

- PROBLEMS
- OUTPUT
- DEBUG CONSOLE
- TERMINAL
- PORTS

Use the bash terminal for this lab.

Note: When you attempt to paste into the VS Code IDE for the first time, it prompts you about this action. Choose **Allow**.

After a few seconds a terminal window appears at the bottom of the developer environment.

21. **Command:** To view configured anomaly detectors, run the following command.



```
aws cloudwatch describe-anomaly-detectors
```

Expected output:.



```
*****
*****  EXAMPLE OUTPUT  *****
*****

{
  "AnomalyDetectors": [
    {
      "Namespace": "TravelApplication",
      "MetricName": "UserLogins",
      "Dimensions": [],
      "Stat": "Average",
      "Configuration": {
        "ExcludedTimeRanges": []
      },
      "StateValue": "TRAINED",
      "SingleMetricAnomalyDetector": {
        "AccountId": "012345678910",
        "Namespace": "TravelApplication",
        "MetricName": "UserLogins",
        "Dimensions": [],
        "Stat": "Average"
```

```

    }
  }
]
}

```

The command output shows the status of the anomaly detector created in the previous task.

22. **Command:** To view the metrics settings of the new alarm, run the following command.



```
aws cloudwatch describe-alarms --alarm-names logins-alarm --query Metr
```

Expected output:.



```

*****
*****  EXAMPLE OUTPUT  *****
*****

```

```

[
  {
    "Id": "m1",
    "MetricStat": {
      "Metric": {
        "Namespace": "TravelApplication",
        "MetricName": "UserLogins",
        "Dimensions": []
      },
      "Period": 60,
      "Stat": "Average"
    },
    "ReturnData": true
  },
  {
    "Id": "ad1",
    "Expression": "ANOMALY_DETECTION_BAND(m1, 2)",
    "Label": "UserLogins (expected)",
    "ReturnData": true
  }
]

```

The alarm uses two metrics. The first metric with an identifier of **m1** is the **UserLogins** metric. The alarm is using the *average* value for this metric over a 60 second period. The second metric uses the ANOMALY_DETECTION_BAND metric math function. ANOMALY_DETECTION_BAND returns an anomaly detection band for the specified metric. The band consists of two time series, one representing the upper limit of the “normal” expected value of the metric, and the other representing the lower limit. The ANOMALY_DETECTION_BAND is passed

two parameters, the first parameter **m1** is a reference to the **UserLogins** metric. The second parameter is the number of standard deviations to use for the band

23. **Command:** To better understand the **ANOMALY_DETECTION_BAND** function, use the **get-metric-data** operation. Run the following command.



```
aws cloudwatch get-metric-data \
  --metric-data-queries ' [{
    "Id": "travelAppLogins",
    "MetricStat": {
      "Metric": {
        "Namespace": "TravelApplication",
        "MetricName": "UserLogins"
      },
      "Period": 60,
      "Stat": "Average"
    },
    "ReturnData": false
  },
  {
    "Id": "combined",
    "Expression": "ANOMALY_DETECTION_BAND(travelAppLogins,2)"
  } ]' \
  --end-time $(date -u +"%Y-%m-%dT%H:%M:%SZ") \
  --start-time $(date -u -d "-5 minutes" +"%Y-%m-%dT%H:%M:%SZ")
```

Note: This command uses the **UserLogins** metric with the **ANOMALY_DETECTION_BAND** function to retrieve metrics calculated with the metric math function for the past five minutes.

Expected output:



```
*****
*****  EXAMPLE OUTPUT  *****
*****
{
  "MetricDataResults": [
    {
      "Id": "combined",
      "Label": "UserLogins Low",
      "Timestamps": [
        "2025-01-29T23:22:00+00:00",
        "2025-01-29T23:21:00+00:00",
        "2025-01-29T23:20:00+00:00",
        "2025-01-29T23:19:00+00:00",
        "2025-01-29T23:18:00+00:00"
      ],
      "Values": [
```

```

                    52.988848987977825,
                    54.301257786043415,
                    54.247587936988891,
                    55.99214426326124,
                    55.323667888547504
                ],
                "StatusCode": "Complete",
                "Messages": []
            },
            {
                "Id": "combined",
                "Label": "UserLogins High",
                "Timestamps": [
                    "2025-01-29T23:22:00+00:00",
                    "2025-01-29T23:21:00+00:00",
                    "2025-01-29T23:20:00+00:00",
                    "2025-01-29T23:19:00+00:00",
                    "2025-01-29T23:18:00+00:00"
                ],
                "Values": [
                    81.4958221041658,
                    82.81010737563017,
                    82.74819654569498,
                    84.47959022533088,
                    83.79417366899693
                ],
                "StatusCode": "Complete",
                "Messages": []
            }
        ],
        "Messages": []
    }
}

```

Note: The results contain both **UserLogins Low** and **UserLogins High** values that define the range of values used for anomaly detection.

Task complete: You successfully retrieved metrics and alarm details with the AWS CLI.

Task 3: Post a custom metric to CloudWatch metrics

In this task, you use a supplied Python script to post custom metrics to CloudWatch. Custom metrics enable you to monitor application-specific data points, such as memory usage by process, that aren't available through default CloudWatch metrics. This level of detailed monitoring is crucial for maintaining application health and troubleshooting performance issues.

The supplied script posts the memory used by nginx and AWS Systems Manager Agent (SSM Agent) every 60 seconds. Having this level of observability into applications makes diagnosing issues, like out of memory exceptions, much easier to scale.

24. In the developer environment, choose the **custom-memory-metrics.py** script.
25. Take some time to read and understand what the code is doing. The script defines a **get_memory_usage_by_process_names** function that is used to retrieve the memory usage of one or more processes. The script uses the **put_metric_data** operation to post memory metrics as a custom metric. **put_metric_data** takes two parameters.
- **Namespace** - CloudWatch uses namespaces as a container for related metrics.
 - **MetricData** - Contains a single instance of metric data. The metric data contains values for:
 - **MetricName** - The name of metric
 - **Dimensions** - A dimension is a name/value pair that is part of the identity of a metric. The script is using the hostname and process name to create a unique name/value pair for the metric.
 - **Value** - The value for the metric. The script uses this to supply the memory usage.
 - **Unit** - The unit the metric is being measured in.

26. **Command:** To run the script, run the following command.



```
python3 custom-memory-metrics.py
```

Note: Allow the script run for about 5 minutes. The script will keep running for the remainder of the lab.

Expected output:



```
*****
***** EXAMPLE OUTPUT *****
*****

posted metric for ip-10-0-1-166.ec2.internal amazon-ssm-agent 18661376
posted metric for ip-10-0-1-166.ec2.internal nginx 10436608
posted metric for ip-10-0-1-166.ec2.internal amazon-ssm-agent 18661376
posted metric for ip-10-0-1-166.ec2.internal nginx 10436608
posted metric for ip-10-0-1-166.ec2.internal amazon-ssm-agent 18661376
posted metric for ip-10-0-1-166.ec2.internal nginx 10436608
```

27. At the top of the AWS Management Console, in the search bar, search for and choose



CloudWatch

28. In the left navigation pane, expand the **Metrics** section, choose **All metrics**.

29. Under **Custom namespaces**, choose **HostResources**.

Note: If you do not see the *Custom namespaces* heading, select the *All* link to show all namespaces including the custom and AWS namespaces.

30. Choose the dimensions **Hostname**, **ProcessName**.

Observe the entries metrics posted from the **custom-memory-metrics.py** script.

31. Select the checkboxes for the two **MemoryUsage** metrics.

Observe the graphed memory values for the **nginx** and **amazon-ssm-agent** processes.

Task complete: You successfully posted custom CloudWatch metrics using a Python script.

Task 4: Observe anomaly detection alarm state

In this task, you remain in the CloudWatch console to observe the state of the anomaly detection alarm created in an earlier task.

32. In CloudWatch console, expand the **Alarms** section, choose **All alarms**.

33. Choose **logins-alarm**.

34. Observe the CloudWatch alarm is now in the alarm state. The graph shows the expected range of values as a gray band. There has been a spike in the UserLogins metric outside of the expected range.

 Anomaly graph

Image description: The diagram depicts a CloudWatch alarm currently in alarm state.

35. Choose **History tab**. The history table displays a time and description for any point in time when an alarm has changed state. This is very helpful for correlating any events that caused the alarm to go into alarm state.

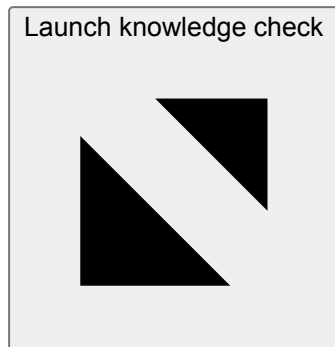
Task complete: You successfully observed a CloudWatch alarm going into a alarm state when the metric went outside the anomaly detection band.

Knowledge Check



This knowledge check is designed to gauge your understanding of the material covered.

Why this matters: Completing this knowledge check will help reinforce your grasp of the content and highlight any areas that may need further review.



Conclusion

You successfully did the following:

- Enabled Amazon CloudWatch anomaly detection for a metric.
- Posted and retrieved Amazon CloudWatch metrics.
- Configured an Amazon CloudWatch anomaly detection alarm.

These skills enable you to implement automated anomaly detection in your applications, reducing the time to identify and respond to unusual behavior. By combining custom metrics with anomaly detection, you can create sophisticated monitoring solutions that automatically adapt to your application's patterns.

End lab

Follow these steps to close the console and end your lab.

36. Return to the **AWS Management Console**.
37. At the upper-right corner of the page, choose **AWS Labs User**, and then choose **Sign out**.
38. Choose **End Lab** and then confirm that you want to end your lab.

Additional resources

- [Using CloudWatch anomaly detection](#)
- API reference for [PutMetricAlarm](#)
- [Collect metrics, logs, and traces with the CloudWatch agent](#)
- VS Code IDE Server is property of Microsoft. Reference in this lab to any specific commercial product, process, or service, or the use of any trade, firm or corporation name is provided for informational purposes, and does not constitute endorsement, recommendation, or favoring by Amazon Web Services.

For more information about AWS Training and Certification, see <https://aws.amazon.com/training/>.

Your feedback is welcome and appreciated.

If you would like to share any feedback, suggestions, or corrections, please provide the details in our [AWS Training and Certification Contact Form](#).

Nice work! Help us make Builder Labs better in just 2 minutes.

[Launch Survey](#)

